

Tema 3 – APIs e Integración de servicios y plataformas – Clase 5

EBIS Business Techschool

Contenidos

- Estructuración de proyectos de integración y conexión con frontends
- Diseñar backend básico
- Uso de APIs con multiples formatos y streaming
- Práctica para llamar a APIs con diferentes formatos



Estructuración de proyectos de integración y conexión con frontends



EBIS Business Techschool

Ya tenemos las piezas: montamos el puzzle

EBIS Business Techschool

Consumir APIs con autenticación

Extraer datos con paginación y filtros

Resistir errores, límites y reintentos

Automatizar flujos y eventos

- Hasta ahora, cada pieza vivía en su script suelto
- Un proyecto real las combina todas… y alguien más tendrá que leerlo y mantenerlo

La caja donde encajan las piezas, y la ventana para que un frontend las vea.

El punto de partida: sync_todo_final_v3.py

EBIS Business Techschool

```
TOKEN = "sk_live_51Ab…"← secreto en el código
url = "https://api.crm.com/v2/…"
# 480 líneas de peticiones,
# transformaciones, emails,
# y prints por todas partes …
```

- Funciona… hasta que hay que cambiar algo
- El secreto viaja con el código: a Git, a los compañeros, a cualquier copia
- Nada se puede probar ni reutilizar por separado
- Nadie más que su autor se atreve a tocarlo

La solución no es escribir mejor código: es darle una estructura.

La estructura del proyecto

EBIS Business Techschool

- **app/** — el código de verdad, organizado por responsabilidad
- **scripts/** — entrypoints que se lanzan a mano o por reloj
- **data/** — lotes descargados, checkpoints, resultados

- **.env** — secretos y configuración local, fuera del código
- **requirements.txt** — las dependencias, para reproducir el entorno

Capas: cada una habla solo con la de abajo



- Las dependencias van en un solo sentido: hacia abajo
- Cambiar de CRM solo toca api_clients; cambiar una regla solo toca services
- Cada capa se puede probar por separado

Variables de entorno y el archivo .env

EBIS Business Techschool

```
.env
CRM_API_TOKEN=sk_live_51Ab…
CRM_BASE_URL=https://api.crm.com/v2
SYNC_PAGE_SIZE=100 LOG_LEVEL=INFO
```

- Pares nombre=valor que viven fuera del código
- En tu máquina los define el .env; en el servidor, el propio entorno
- El mismo código funciona en desarrollo y producción con valores distintos

Regla inquebrantable: el .env va en el `.gitignore` . Los secretos nunca entran en Git — se comparte un .env.example con los nombres, sin los valores.

config.py: la única puerta a la configuración

EBIS Business Techschool

app/config.py

```
import os CRM_API_TOKEN =

os.getenv("CRM_API_TOKEN")

CRM_BASE_URL = os.getenv("CRM_BASE_URL")

PAGE_SIZE = int(os.getenv("SYNC_PAGE_SIZE", "100")) if not \

CRM_API_TOKEN: raise RuntimeError("Falta CRM_API_TOKEN")
```

- Un solo archivo lee el entorno; el resto importa de aquí
- Valores por defecto sensatos donde tenga sentido
- Si falta un secreto, el proyecto falla al arrancar — no a mitad de una sincronización

El cliente API: toda la técnica, encapsulada

EBIS Business Techschool

```
app/api_clients/external_api.py
class CrmClient:
    def get_contactos(self, desde):
        # autenticación, paginación,
        # reintentos y timeouts
        # viven SOLO aquí …
```

- Aquí viven las clases 1 a 3: tokens, paginación, backoff, timeouts
- Quien lo usa pide «dame los contactos» y recibe datos limpios
- Un archivo por API externa: crm.py, erp.py, slack.py…
- Si el proveedor cambia su API, solo se toca este archivo

Los servicios: las reglas del flujo

EBIS Business Techschool

```
app/services/sync_service.py
def sincronizar_contactos():
    nuevos = crm.get_contactos(desde=checkpoint)
    validos = validar(nuevos)
    filas = mapear_campos(validos)
    hoja.upsert(filas)
    guardar_checkpoint()
```

- Combina clientes y aplica las reglas: validación, mapeo, upsert, checkpoint
- Se lee como la descripción del flujo, casi en lenguaje natural
- No sabe nada de HTTP ni de tokens: eso queda en los clientes

Los scripts: el botón de encendido

EBIS Business Techschool

scripts/run_sync.py

```
from app.services
import sync_service
if __name__ == "__main__":
    sync_service.sincronizar_contactos()
```

- Mínimo a propósito: llama al servicio y termina
- Se lanza a mano, por reloj (cron) o desde el orquestador
- Si crece más de unas pocas líneas, esa lógica pertenece a un servicio

Un proyecto puede tener varios scripts: run_sync.py, run_informe.py… todos igual de finos.

utils/logger.py: un logger para todo el proyecto

EBIS Business Techschool

- Se configura una vez: formato con fecha y módulo, nivel leído de config
- Todos los módulos lo importan: mismo formato en todo el proyecto
- Escribe a archivo (un log por día) o a la salida estándar en producción

```
2026-07-05 07:00:01 INFO [sync_service] inicio: 240 contactos desde el checkpoint
2026-07-05 07:00:14 WARN [crm_client] 429 recibido, espero 30 s (intento 1/5)
2026-07-05 07:01:02 INFO [sync_service] fin: 238 filas, 2 apartados a revisión
```

- Prohibido el print suelto: lo que no pasa por el logger no existe cuando algo falla

Tests básicos: los tres que valen oro

EBIS Business Techschool

Transformaciones

¿El mapeo convierte bien fechas, importes y nombres? Entrada conocida, salida esperada.

Validaciones

¿El email inválido se aparta? ¿El campo vacío se rechaza? Los datos sucios, cubiertos.

Cliente con respuestas simuladas

¿Recorre bien las páginas? ¿Reacciona bien a un 429? Sin tocar la API real.

La estructura en capas es lo que hace esto posible: cada pieza se prueba sola.

Cambio de rol: de consumir a servir

EBIS Business Techschool



- El frontend nunca habla con el CRM o el ERP directamente: ni sus tokens ni su complejidad
- Tu API es la fachada: expone justo lo que el panel necesita, ya combinado y limpio
- Todo lo que sabes como consumidor de APIs ahora lo aplicas como diseñador de una

FastAPI: exponer datos con poco código

EBIS Business Techschool

Ligero

Un endpoint son unas pocas líneas en main.py

Valida solo

Parámetros y tipos se comprueban automáticamente

Se documenta solo

Documentación interactiva gratis en /docs

- Encaja en la estructura: `app/main.py` es la puerta de entrada de la API
- Los endpoints llaman a los mismos servicios que usan los scripts: cero duplicación

main.py: tu primer endpoint

EBIS Business Techschool

```
app/main.py
from fastapi import FastAPI
from app.services
import sync_service
app = FastAPI()
@app.get("/ventas/resumen ")
def resumen_ventas():
    return sync_service.resumen_ventas()
```

- La ruta declara qué URL responde: GET /ventas/resumen
- La función devuelve datos; FastAPI los convierte a JSON
- El endpoint no calcula nada: delega en el servicio, como los scripts
- En /docs aparece ya documentado y se puede probar desde el navegador

Diseñar la respuesta para el frontend

EBIS Business Techschool

```
GET /ventas/resumen
{ "total_mes": 48250.0, "nuevos_clientes": 17,
"pendientes": 4, "ultima_sincronizacion":
"2026-07-05T07:01:02Z" }
```

- A medida de la pantalla: los tres números que el panel va a pintar
- Nombres claros y unidades decididas: nada de céntimos sorpresa
- La fecha de última sincronización: el frontend puede avisar si los datos son viejos
- Si la lista crece, ya sabes qué toca: paginarla, como harían contigo

Tres detalles antes de conectar el frontend

EBIS Business Techschool

CORS

El navegador bloquea llamadas entre dominios distintos: hay que autorizar el dominio del panel explícitamente.

Autenticación

Tu API también necesita su llave: una API key interna como mínimo, aunque solo la use tu panel.

Errores coherentes

Ahora eliges tú los códigos: 404 si no hay datos, 500 si algo falló — con un JSON de error claro.

Todo lo que exigías como consumidor de APIs, ahora te toca ofrecerlo.

Monitorizar

El log cuenta qué pasó. Monitorizar es saber cómo está el sistema ahora — y que te avise él.

¿Se está ejecutando?

La sincronización de las 7:00, ¿corrió hoy? ¿Terminó?

¿Cuánto procesa?

Registros por ejecución: 240 ayer, 0 hoy — algo pasa

¿Cuántos errores?

Tasa de fallos por ejecución y su tendencia

¿Cuánto tarda?

Duración y tiempos de respuesta: lo lento avisa antes de romperse

Métricas simples: un resumen por ejecución

EBIS Business Techschool

```
{ "ejecucion": "2026-07-05T07:00Z", "duracion_s": 61, "procesados": 238, "errores": 2, "reintentos":
}
```

- Contadores que ya tienes: procesados, errores, reintentos, duración
- Guardado como línea de log o archivo en data/: sin herramientas nuevas
- La tendencia vale más que el valor: 238 hoy es normal; 0, una alarma

Con cinco números por ejecución se responde el 90% de «¿va todo bien?».

De los errores a las alertas



- Un log que nadie mira no protege nada: el ERROR debe llegar solo a una persona
- Alerta solo lo accionable: si avisa de todo, se ignora todo
- También alerta la ausencia: «hoy no corrió la sincronización» es la peor avería silenciosa

El endpoint /health: el pulso del sistema

```
GET /health
{ "status": "ok", "ultima_sync":
"2026-07-05T07:01Z", "errores_hoy": 2 }
```

- Tu API declara su propio estado en un endpoint fijo
- Un vigilante externo lo consulta cada minuto; si no hay 200, avisa
- Puede incluir chequeos internos: ¿hay checkpoint reciente? ¿responde la BD?

Quiz 1

El bucle de paginación con reintentos para leer del CRM, ¿en qué capa vive?

A En el script run_sync.py, que es quien lanza la tarea

B En api_clients/: es técnica de hablar con esa API

C En services/, junto a las reglas de la sincronización

Quiz 2

Un panel web necesita mostrar ventas del CRM y stock del ERP. ¿De dónde saca los datos?

A Llama al CRM y al ERP directamente desde el navegador

B Lee los archivos de data/ directamente del servidor

C Consume un endpoint de tu API propia, con los datos ya combinados

EBIS Business Techschool

Quiz 3

La sincronización corre cada día sin errores… pero lleva una semana procesando 0 registros. ¿Qué lo habría detectado?

A Las alertas de ERROR: avisarían del fallo

B El /health: devolvería un error al vigilante

C La métrica de procesados por ejecución y su tendencia

EBIS Business Techschool

Checklist del proyecto profesional

EBIS Business Techschool

- Capas separadas: clientes, servicios, scripts, config
- config.py como única puerta a la configuración
- Tests para transformaciones, validaciones y cliente
- Métricas por ejecución: procesados, errores, duración
- Secretos en .env, y el .env en el .gitignore
- Logger común; ningún print suelto
- Endpoints diseñados a medida del frontend, con CORS y llave
- Alertas de ERROR y de ausencia; /health vigilado

Si el proyecto cumple los ocho, cualquiera del equipo puede mantenerlo — no solo su autor.

Diseñar backend básico



EBIS Business Techschool

Creciendo la api básica a backend funcional

Vamos a añadir a la api básica que ya tenemos, modelos y endpoints típicos de un backend

Uso de APIs con múltiples formatos



EBIS Business Techschool

Content-Type: la etiqueta del paquete

EBIS Business Techschool

HTTP puede transportar cualquier cosa. La cabecera Content-Type declara qué hay dentro del cuerpo.

`application/json`
Lo que llevamos usando todo el curso

`image/png` · `image/jpeg`
Imágenes: bytes crudos del archivo

`audio/mpeg` · `video/mp4`
Audio y vídeo, enteros o en trozos

`application/pdf`
Documentos: facturas, informes, contratos

`application/octet-stream`

Binario genérico: «bytes sin más»

`multipart/form-data`
Varios campos + archivos en una petición

Primera regla: mirar siempre el Content-Type antes de intentar interpretar la respuesta.

Descargar un archivo: bytes, no texto

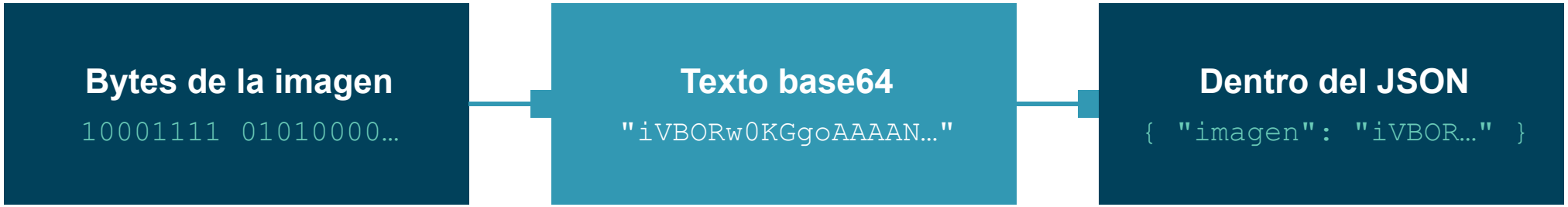
```
r = requests.get(url_imagen)
r.json() ← solo si es JSON
r.text ← texto: corrompe binarios
r.content ← bytes: lo correcto aquí
with open("foto.png", "wb ") as f:
    f.write(r.content)
```

- Un archivo son bytes: se lee con .content y se guarda en modo binario («wb»)
- El error clásico: tratarlo como texto y obtener un archivo corrupto
- Para archivos grandes: descargar en trozos (stream) para no llenar la memoria — como la paginación, pero de bytes

Base64: binario disfrazado de texto

EBIS Business Techschool

JSON solo admite texto. ¿Cómo meter una imagen dentro? Codificándola como una cadena.



- Se decodifica al recibir: `base64.b64decode(cadena)` → bytes → archivo
- Coste: ocupa un 33% más que el binario original — solo para archivos pequeños o medianos
- Lo verás en APIs de imágenes, adjuntos de email y webhooks que llevan archivos

Subir archivos: multipart/form-data

EBIS Business Techschool

```
requests.post(
     url,
     data={"titulo": "Factura julio"},
     files ={
          "archivo": open("factura.pdf", "rb ")
     }
)
```

- Una sola petición mezcla campos de texto y archivos
- El archivo se abre en modo binario («rb»), igual que se guardaba en «wb»
- Casos típicos: subir un CSV a procesar, un audio a transcribir, una imagen de producto
- Las APIs limitan el tamaño: revisa el máximo en la documentación antes de subir

Streaming: contenido que llega en flujo

EBIS Business Techschool

Descarga por trozos

Un vídeo de 2 GB se lee y guarda en bloques: la memoria nunca se llena.

Flujo en directo

Audio o vídeo que no termina: se procesa cada fragmento según llega.

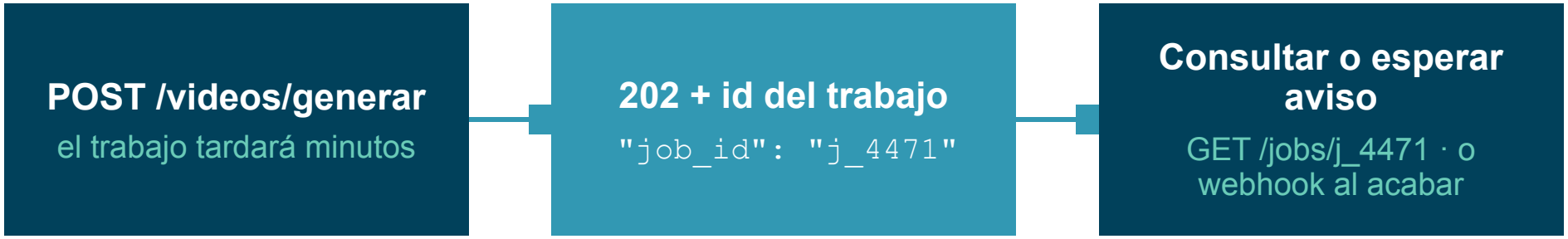
Respuestas de IA en vivo

El texto llega palabra a palabra (eventos SSE) en vez de esperar la respuesta entera.

- Idea común: procesar según llega, sin esperar a tenerlo todo
- La conexión queda abierta mucho más tiempo: los timeouts se configuran distinto

Trabajos largos: te doy un ticket, no el resultado

EBIS Business Techschool



- El 202 significa «aceptado, en proceso»: el resultado vendrá después
- Dos formas de enterarte: preguntar por el estado (polling) o recibir un webhook — ambas ya las dominas
- Al terminar, la respuesta trae la URL del archivo generado para descargarlo

Este patrón aparece en generación de vídeo, exportaciones grandes y procesado de documentos.

Una API de IA es… una API

EBIS Business Techschool

Todo lo conocido aplica

Token en la cabecera, POST con JSON, códigos de error, rate limits, reintentos con backoff, timeouts generosos.

Lo nuevo

El input es una descripción en lenguaje natural (prompt) y el output es un archivo binario: la imagen.

- Proveedores habituales: OpenAI (gpt-image), Stability AI, Google (Imagen), Ideogram…
- Mismo esquema en todos: cambia el nombre de los parámetros, no el patrón

El prompt como input estructurado

EBIS Business Techschool

```
plantilla de prompt
"Foto de producto de {nombre_producto} sobre fondo {color_marca} , iluminación de estudio, estilo minimalista, sin texto"
```

- En automatización, el prompt es una plantilla: los huecos se rellenan con datos del negocio
- Fija el estilo en la plantilla (iluminación, fondo, tono): la variación va solo en los huecos
- Se versiona y se prueba como el código: la plantilla v3 «funciona»; no se toca sin comparar

El prompt pertenece a la capa de servicios: es una regla de negocio, no técnica de API.

La petición: prompt + parámetros

EBIS Business Techschool

```
POST /v1/images/generations {
"prompt": "Foto de producto de…",
"size": "1024x1024",
"style": "natural",
"n": 3,
"response_format": "url"
}
```

- **size:** resolución — cuadrada, horizontal o vertical según destino
- **style / quality:** foto natural, ilustración, mayor o menor detalle
- **n:** número de variantes por petición — para elegir la mejor
- **response_format:** URL temporal o base64 dentro del JSON

- Cada parámetro afecta al precio: más resolución, más calidad y más variantes = más coste

La respuesta: URL temporal o base64

EBIS Business Techschool

Formato URL

```
{ "url": "https://…/img.png?exp=2h" }
```

Enlace temporal: se descarga con una petición más (bytes, «wb»). Caduca en horas.

Formato base64

```
{ "b64_json": "iVBORw0KGgo…" }
```

La imagen viaja dentro del JSON: se decodifica a bytes y se guarda. Respuestas más pesadas.

Regla de oro: descarga y guarda siempre en tu almacenamiento. La URL del proveedor no es un archivo: es un préstamo con caducidad.

Caso 1 · Creatividades para marketing



- De días de ida y vuelta con diseño a horas: la agencia revisa en vez de producir
- La persona no desaparece: la IA propone, el humano aprueba — siempre

Casos 2 y 3 · Producto y prototipos

EBIS Business Techschool

Imágenes para productos

- Variantes de fondo y contexto para cada artículo del catálogo
- Disparado por webhook: al dar de alta un producto, se generan sus imágenes
- Datos del ERP rellenan la plantilla: nombre, categoría, color

Prototipos visuales

- Conceptos y moodboards antes de invertir en diseño o fotografía real
- Diez direcciones visuales en una tarde para decidir con el cliente
- Lo elegido pasa a producción con diseñadores: la IA explora, no entrega

Casos 4 y 5 · Redes sociales y generación masiva

EBIS Business Techschool

Assets para redes sociales

- El calendario de contenidos dispara la generación semanal
- Un concepto, varios tamaños: cuadrado, story vertical, banner
- Publicación programada vía la API de cada red, tras aprobación

Generación masiva controlada

- Miles de imágenes en lotes, con pausas y respeto al rate limit
- Presupuesto máximo por ejecución: el bucle se detiene al alcanzarlo
- Muestreo humano: revisar un porcentaje de cada lote antes de seguir

«Masiva» sin «controlada» es una factura sorpresa y una marca en riesgo.

Nada nuevo bajo el sol: es una automatización más

EBIS Business Techschool

- **Entrada:** datos del negocio (catálogo, brief, calendario) + un disparador ya conocido
- **Transformación:** plantilla de prompt + llamada a la API + validación del resultado
- **Salida:** archivo + ficha en tu almacenamiento, y publicación vía otra API
- **En el proyecto:** el cliente de la API de imágenes vive en api_clients/; la plantilla, en services/

Todo el curso aplica tal cual: solo ha cambiado lo que viaja en el cuerpo de la respuesta.

Costes: aquí cada petición es dinero

```
1.000 productos × 4 variantes × 0,04 € = 160 € …con un bucle que reintenta mal: × 5 =
800 €
```

- Presupuesto máximo por ejecución, contado en el propio bucle: al llegar, parar
- Alertas de gasto en el panel del proveedor, antes de que llegue la factura
- Probar con tamaño mínimo y calidad baja; producción es el último paso
- El coste va en la ficha de cada imagen: el gasto se audita como los datos

Privacidad y derechos de uso

EBIS Business Techschool

Lo que envías

- El prompt viaja a un tercero: sin datos de clientes ni secretos de producto
- Revisar la política del proveedor: ¿retiene los prompts? ¿entrena con ellos?
- Las fotos subidas para editar cuentan como datos personales si hay personas

Lo que recibes

- El derecho a uso comercial depende del proveedor y del plan: leerlo antes
- La imagen puede parecerse a marcas, obras o personas reales: revisar antes de publicar
- Algunos sectores exigen etiquetar el contenido generado por IA

Moderación: el doble filtro

EBIS Business Techschool

El filtro del proveedor

Rechaza prompts que violan sus políticas y devuelve un error específico. Es un error **no recuperable** : reintentarlo dará lo mismo — se registra y se aparta.

Tu propio filtro

En un flujo automático nada llega al público sin validación: revisión humana, lista de términos prohibidos en prompts, muestreo de lotes.

Riesgo especial: texto de usuarios que acaba dentro de un prompt. Se valida antes, como cualquier dato sucio.

Trazabilidad: cada imagen, con su historia

La pregunta que llegará tarde o temprano: «esta imagen publicada, ¿de dónde salió?»

- Qué prompt y qué parámetros la generaron, con qué modelo y versión
- Cuándo se generó, cuánto costó, en qué ejecución del flujo
- Quién la aprobó y dónde se publicó

La ficha de metadatos + el log de aprobaciones responden en segundos lo que sin ellos es imposible reconstruir.

Quiz 1

Descargas un PDF desde una API con requests. ¿Cómo lo guardas correctamente?

A Escribir r.text en un archivo en modo “w” con extensión .pdf obligatoria

B Escribir r.content en un archivo abierto en modo «wb»

C Decodificar r.text con base64 y guardar el resultado en el archivo

EBIS Business Techschool

Quiz 2

La API te devuelve las imágenes como URLs que caducan en 2 horas. ¿Qué hace tu flujo?

A Usar las URLs directamente en la web de la empresa

B Guardar las URLs en la base de datos para descargarlas solo si hacen falta

C Descargar los bytes ya y guardarlos en tu almacenamiento

Quiz 3

En un lote nocturno, la API rechaza un prompt por política de contenido. ¿Qué hace el flujo?

A Reintentar con backoff exponencial hasta que lo acepte

B Registrarlo, apartarlo a revisión humana y seguir con el lote

C Reformular el prompt automáticamente y publicar el resultado

EBIS Business Techschool

Checklist de generación automatizada

EBIS Business Techschool

- Binarios con .content y «wb»; base64 se decodifica
- Descargar siempre: la URL del proveedor caduca
- Presupuesto por ejecución y alertas de gasto
- Derechos de uso comercial verificados en el plan contratado
- Prompt como plantilla versionada, en la capa de servicios
- Cada imagen con su ficha: prompt, parámetros, coste
- Sin datos de clientes ni secretos en los prompts
- Nada se publica sin validación o revisión humana

Sirve para imágenes hoy — y para audio, vídeo o cualquier archivo generado mañana.

Práctica para llamar a APIs con diferentes formatos



EBIS Business Techschool

Llamando a una api con diferentes formatos

Vamos a enviar y recibir imagenes, audio, pdfs, texto en streaming y audio en streaming



EBIS Business Techschool

www.ebiseducation.com